

SECURE DIGITAL BANKING PLATFORM

Milestone 2 — Detailed Project Report

Core Focus: Loan origination • Credit check • EMI calculation | Team C

1. Introduction

Secure is a modular digital banking platform built with an Angular frontend, a Spring Boot REST backend, and a MySQL/PostgreSQL data layer. Milestone 2 extends the platform with a full Loan Management suite: loan origination (a six-stage workflow from pre-qualification through funding), automated credit assessment with a rule-based eligibility engine, and EMI (Equated Monthly Installment) calculation. This report documents the three folders delivered for this milestone — frontend-milestone2, backend-M2, and DB — at the level of routes, entities, DTOs, REST endpoints, validation rules, business logic, and SQL/PHP scripts, based on a direct read of the source on the team-c-milestone-2 branch.

Sections 4–6 go module-by-module through the actual code (entities, services, validators) rather than summarizing at a high level, since the goal of this report is to give reviewers enough detail to assess correctness without re-reading the repository themselves.

2. Technology Stack

Layer	Technology	Detail
Frontend	Angular 19.x	Standalone components + one routed NgModule (Loan Origination); Angular Material, Angular CDK, RxJS 7.8, Chart.js 4.4
Backend	Java 17 / Spring Boot	Spring Web (REST), Spring Data JPA, Lombok, Maven (mvnw wrapper committed)
Database (backend-M2)	PostgreSQL	Database "Banking" @ localhost:5432, user postgres, Hibernate ddl-auto=update
Database (DB/ folder)	MySQL	Database "digital_banking", managed via XAMPP / phpMyAdmin, port 3306
Legacy data-access	PHP (mysqli)	3 procedural scripts mirroring the loan workflows against MySQL, using prepared statements
Version Control	Git & GitHub	Branch: team-c-milestone-2

3. What Was Delivered

The milestone delivery adds three top-level folders to the existing repository, on top of the Milestone-1 frontend/backend/Screenshots already present:

- **frontend-milestone2/**: Angular 19 workspace, single app, 3 routed feature areas, ~90 source files, ~7,850 lines of TS/HTML/SCSS.
- **backend-M2/**: Spring Boot service exposing 11 REST endpoints across 3 packages, 27 Java files, ~2,740 lines.

- **DB/:** 4 SQL scripts (~380 lines) defining/seeding/querying the MySQL schema, plus 3 PHP scripts (~560 lines) as an alternate data-access layer.

4. Frontend

Built on Angular 19 with a hybrid architecture: the Loan Origination feature is a lazily-loaded NgModule with its own routing module, while Credit Check and EMI Calculator are lazily-loaded standalone components declared directly in app.routes.ts. The root route redirects "" to /loan-origination, and an unmatched wildcard route also falls back to /loan-origination.

4.1 App Navigation

Path	Loads	Type
/ (default)	redirects to /loan-origination	redirect
/loan-origination	LoanOriginationModule (lazy)	NgModule + child routes
/credit-check	CreditCheckDashboardComponent	standalone, lazy
/emi-calculator	EmiCalculatorComponent	standalone, lazy
** (wildcard)	redirects to /loan-origination	redirect

4.2 Loan Origination — The Main Workflow

Implemented as loan-origination.module.ts with loan-origination-routing.module.ts, and a dedicated app-shell component that hosts a persistent header-bar and sidebar around the routed page content. Eight page-level components implement the following pipeline:

- Dashboard Page — portfolio overview / landing page for the module
- Pre-Qualification Page — initial eligibility screening before a full application is opened
- Loan Application Page — full applicant + loan detail capture form (personal, address, employment, income, loan terms)
- Application Processing Page — tracks an application while it is in flight, likely surfacing a progress-tracker component
- Credit Check Page — embeds the credit evaluation step inside the origination workflow
- Underwriting Page — decisioning step
- Quality Control Page — checklist-driven review, backed by the reusable quality-checklist component (items typed as {label, status: 'Pending'|'Passed'|'Failed'})
- Loan Funding Page — disbursal step, paired with a funding-summary-card component

Reusable UI building blocks under components/: app-shell, header-bar, sidebar, progress-tracker, application-card, funding-summary-card, quality-checklist. A shared status-badge component (in shared/components) plus a status-class.pipe.ts centralize status → colour/class mapping so every page renders application/credit/funding statuses consistently.

4.4 EMI Calculator

A single standalone component (emi-calculator.component.ts/.html/.scss) backed by emi.service.ts and emi-calculation.model.ts, calling POST /api/emi/calculate to obtain monthly EMI, total interest and total payable for a given principal, annual interest rate and tenure.

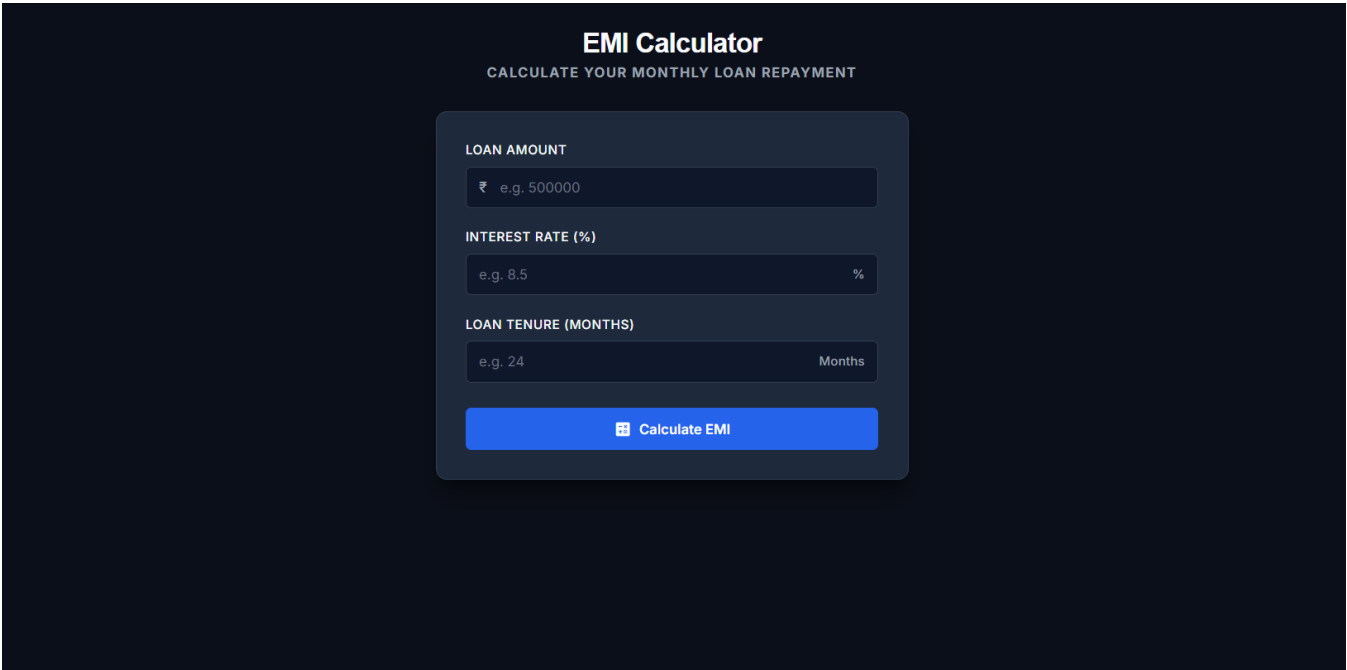


Fig 3- EMI Calculator

4.5 Form Validation Rules

A CustomValidators class provides reusable Angular reactive-form validators with concrete regex-backed rules, used across the Loan Application and related forms:

Validator	Rule
name()	/^[A-Za-z]{2,60}\$/ — letters and spaces only, 2–60 characters
mobile()	/^[6-9]\d{9}\$/ — 10-digit Indian mobile number starting 6-9; also rejects repeated-digit numbers (e.g. 9999999999) as "unrealistic"
dateOfBirth()	Rejects future dates and computes exact age; requires age between 18 and 120
income()	Digits only, must be > 0 and ≤ 100,000,000
deposit(min)	Up to 2 decimal places, must be ≥ a caller-supplied minimum
nomineeNotSelf(fullnameControl)	Cross-field: nominee name (case-insensitive) must differ from the account holder's own name
passwordsMatch(passwordControl)	Cross-field: confirm-password must equal the password control's value

5. Backend

A single Spring Boot application (BankingManagementApplication) organized into three feature packages under com.Secure.BankingManagement, each following a Controller → Service → Repository → Entity/DTO layering. All controllers are annotated @CrossOrigin to allow the Angular dev server to call them directly.

5.1 API Endpoints

Module	Method & Path	Purpose
Loan Origination	POST /api/loan-origination	Create a loan application (201/400/500)
Loan Origination	GET /api/loan-origination	List all applications
Loan Origination	PUT /api/loan-origination/{loanId}/status?status=	Transition an application's status (404 if missing)
Credit Check	GET /credit-check(s)	List all saved credit checks
Credit Check	POST /credit-check(s)	Run eligibility AND persist a CreditCheck row
EMI	POST /api/emi/calculate	Calculate & persist an EMI breakdown for a loan (201)

5.2 Loan Origination

What a Loan Application Stores

This is the entity that was substantially enhanced this milestone (per the team's own LOAN_ORIGINATION_COMPLETION.md). Field groups, exactly as declared:

Group	Fields
Identifiers	loanId (PK, identity), customerId (not null), customerName (not null)
Personal	fullName, dateOfBirth (LocalDate), gender (≤20), mobile (≤20), email (≤100)
Address	address (≤255), city (≤100), state (≤100), pincode (≤20)
Employment	employmentType (≤50), employerName (≤255), jobTitle (≤100), workExperience (≤100)
Income	monthlyIncome (decimal 15,2), otherIncome (decimal 15,2)
Loan terms	loanType (enum, not null), loanAmount (decimal 15,2, not null), tenureMonths (Integer, not null), interestRate (decimal 5,2, not null), purpose (≤500)
Status / audit	applicationStatus (enum, not null), applicationDate (LocalDate), createdAt (LocalDateTime, not updatable), updatedAt (LocalDateTime)

Two JPA lifecycle callbacks give the entity sensible defaults without relying on the caller: @PrePersist sets createdAt/updatedAt to now(), defaults applicationDate to today if absent, and defaults applicationStatus to PENDING if absent; @PreUpdate refreshes updatedAt on every save.

Fixed Value Lists

The following two lists (enums) constrain what values loanType and applicationStatus can take:

- **LoanType:** Personal, Home, Vehicle, Education, Gold, Other

- **ApplicationStatus (LoanOrigination package):** DRAFT, PENDING, UNDER_REVIEW, APPROVED, REJECTED, FUNDED

Request & Response Formats

The request DTO accepts both loanAmount and requestedAmount (the frontend's older field name), and both fullName and customerName, so the service layer can normalize whichever the caller sends. customerId is optional — a loan can be created without an existing Customer row, which the service backs up by generating a placeholder customerId internally when absent. The response DTO (built with the Lombok Builder pattern) mirrors every entity field plus createdAt/updatedAt for the client.

How the Service Handles Requests

- Parses string dates from the frontend into LocalDate, tolerating missing/blank values.
- Falls back application status to PENDING if the incoming string doesn't match a known ApplicationStatus.
- Looks up the interest rate automatically via LoanProductService based on loanType when the caller doesn't supply one.
- convertToResponse() / convertToResponseList() centralize entity → DTO mapping for every read endpoint.

5.3 Credit Check & Eligibility Rules

Customer Records

A plain-Java (non-Lombok, hand-written getters/setters) entity: customerId (PK), fullName, email, phone, dateOfBirth, gender, mobileNo, emailId, address, employmentType (column e_type), salary, createdAt, timeStamp. Note the entity carries both email/phone and emailId/mobileNo pairs — likely a legacy duplication from an earlier iteration of the schema.

A Second Loan Record (Credit Check Side)

A second, independent JPA mapping onto the same loan_application table used by the CreditCheck package (distinct from LoanOrigination's own entity, which maps to a different table, loan_origination). Fields: loanId (Integer PK), a @ManyToOne Customer, loanType (String, not the LoanOrigination enum), loanAmount, tenureMonths, interestRate, purpose, applicationStatus (its own ApplicationStatus enum, defaulting to PENDING), applicationDate, plus @OneToMany collections back to CreditCheck, EmiCalculation and LoanHistory rows for that loan.

Request & Response Formats

CreditCheckRequest carries customerId, customerName, loanId (optional — omitting it creates a new loan application on save), loanType, loanAmount, monthlyIncome, creditScore, existingLoanCount, creditStatus, remarks. CreditCheckResponse mirrors this plus creditCheckId, checkedAt and a human-readable message field. CustomerLookupResponse (used by GET /credit-check/customer/{id}) returns the customer's profile, most recent loan (if any), and their full previous-loan history as a list of PreviousLoanResponse — giving the frontend everything it needs to pre-fill a new credit check form for a returning customer.

How Eligibility Is Decided

This is the most substantial piece of business logic in the milestone. Given a customer, requested loan amount/type and monthly income, it:

- Loads the customer's full loan_history and counts existingLoanCount and sums outstanding balances (outstandingAmount) across every historical loan with a positive balance.

- Flags hasDefault = true if any historical loan has loan_status = DEFAULTED.
- Estimates a monthly EMI for the requested amount using a flat assumed 12% annual rate over a flat assumed 36-month tenure (independent of the loan's actual requested tenure), via the standard amortization formula.
- Computes a debt-to-income ratio: (outstanding balance ÷ 36 + estimated EMI) ÷ monthly income.

The five decision rules are evaluated in this order (first match wins):

#	Condition	Result	Remark generated
1	Any historical loan is DEFAULTED	FAIL	"Customer has a defaulted loan on record..."
2	creditScore < 600	FAIL	"Credit score (...) is below the minimum threshold of 600."
3	existingLoanCount ≥ 4	FAIL	"Customer already holds N loans, exceeding the maximum allowed."
4	debt-to-income > 60%	FAIL	"Debt-to-income ratio is X%, exceeding the 60% cap."
5	creditScore < 700, or DTI > 40%, or existingLoanCount > 0	REVIEW	one of three messages depending on which sub-condition triggered
—	none of the above	PASS	"Meets standard lending criteria. No further review required."

Saving a Credit Check

POST /credit-check(s) both evaluates eligibility and writes a record: if no loanId is supplied it first creates a new LoanApplication (36-month tenure, flat 12.00% interest rate, purpose auto-set to "<LoanType> loan requirement", status PENDING), otherwise it re-uses the existing loan by ID (throwing 404 if not found). It then builds and saves a CreditCheck row carrying the computed existingLoanCount, previousLoanStatus (YES/NO based on whether any history exists), and the PASS/REVIEW/FAIL outcome.

5.4 EMI Calculation

A self-contained module: EMICalculationController exposes a single POST /api/emi/calculate endpoint, delegating to EMICalculationService.calculateEMI(). The service:

- Validates the request: loanId required; principalAmount must be > 0; interestRate must be ≥ 0; tenureMonths must be > 0 — each violation throws IllegalArgumentException with a specific message.
- Converts the annual interest rate to a monthly rate (rate ÷ 1200) using BigDecimal division at 10-digit scale.
- Special-cases a zero interest rate: EMI = principal ÷ tenureMonths (simple division, no amortization).
- Otherwise applies the standard amortization formula $P \times r \times (1+r)^n / ((1+r)^n - 1)$, computing the power term via double-precision Math.pow() and everything else in BigDecimal, rounding EMI/total interest/total payable to 2 decimal places with HALF_UP rounding.
- Persists the result to the emi_calculation table via EMICalculationRepository and returns an EMICalculationResponse with emild, principal, rate, tenure, monthlyEmi, totalInterest, totalPayable and calculatedAt.

Because the power term is computed in double precision before being converted back to BigDecimal, extremely long tenures or rates could accumulate small floating-point rounding error — acceptable for a 2-decimal-place monetary display, but worth flagging if the platform later needs audit-grade precision.

5.5 Database Connection Settings

application.properties targets PostgreSQL — database "Banking" on localhost:5432 (user postgres), with Hibernate ddl-auto set to update, meaning entity changes are applied to the schema automatically on each application start rather than through versioned migrations.

6. Database

The DB folder documents the MySQL schema (digital_banking database) that backs the loan feature set independently of the Spring Boot/PostgreSQL path above, plus a PHP data-access layer that performs the same three operations directly against MySQL — useful for testing via XAMPP/phpMyAdmin without running the Java backend at all.

6.1 Table Structure

Table	Key Columns	Constraints / Enum Values
loan_application	loan_id (PK, AI), customer_id (FK→customer), loan_type, loan_amount DECIMAL(15,2), tenure_months, interest_rate DECIMAL(5,2), purpose, application_status, application_date	loan_type ENUM(Personal,Home,Vehicle,Education,Gold,Other); application_status ENUM(Pending,Approved,Rejected) DEFAULT Pending
loan_history	history_id (PK, AI), customer_id (FK), loan_id (FK, nullable), loan_type, loan_amount, outstanding_amount DEFAULT 0.00, loan_status, start_date, end_date, created_at	loan_status ENUM(Active,Closed,Defaulted)
credit_check	credit_check_id (PK, AI), loan_id (FK), credit_score, monthly_income, existing_loan_count DEFAULT 0, previous_loan_status, credit_status, remarks, checked_at	previous_loan_status ENUM(Yes,No) DEFAULT No; credit_status ENUM(Pass,Review,Fail) DEFAULT Review
emi_calculation	emi_id (PK, AI), loan_id (FK), principal_amount, interest_rate, tenure_months, monthly_emi, total_interest, total_payable, calculated_at	All monetary columns DECIMAL(15,2) except interest_rate DECIMAL(5,2)

All four tables carry a foreign key back to loan_application (or to customer), so the schema enforces that credit checks and EMI calculations can never exist without a parent loan — a stricter invariant than the JPA/PostgreSQL side currently guarantees for the optional-customerId case in Section 5.2.

6.2 Sample Data

Seeds 2 loan applications (customer 1: ₹200,000 Personal loan at 10.50% for 24 months, Pending; customer 2: ₹1,500,000 Home loan at 8.50% for 120 months, Approved), a closed Vehicle loan history row and an active Personal loan history row (with a ₹50,000 outstanding balance), and matching credit_check rows — enough sample data to exercise every query in 08_credit_check.sql without a running application.

6.3 Example Queries

Five hand-written SELECT queries documenting how the credit-check workflow reads the schema directly, useful as a spec for anyone re-implementing the logic outside Spring Boot:

- Previous loans for a customer_id, most recent first (implicitly, by omission of ORDER BY it returns in insertion order).
- Active/existing loan count for a customer via COUNT(*) ... WHERE loan_status = 'Active' GROUP BY customer_id.
- Credit-check details for a given loan_id.
- A 3-way join (customer ⋈ loan_application ⋈ credit_check) returning a full credit-validation view for a loan.
- A previous-loan existence check using LEFT JOIN + CASE, returning 'Yes'/'No' per customer — the SQL-level equivalent of the Java service's existingLoanCount > 0 check.

6.4 PHP Scripts

Three procedural PHP scripts (mysqli, prepared statements throughout) mirror the same three workflows against MySQL, independent of the Java backend:

Creating a Loan Application

On POST: reads customer_id, loan_type, loan_amount, tenure_months, interest_rate, purpose from \$_POST; validates all required fields are non-empty, loan_amount > 0, and tenure_months > 0 (die()-ing with a message otherwise) before inserting a new loan_application row.

Recording a Credit Check

On POST: reads loan_id, credit_score, monthly_income; validates all three are present, credit_score is within 300–900, and monthly_income > 0; then looks up the owning customer_id from loan_application via a prepared SELECT before recording the credit check — the same customer-resolution pattern used by the Java CreditCheckServiceImpl.

Looking Up a Loan for EMI

On GET: requires loan_id, fetches loan_id/loan_amount/interest_rate/tenure_months/application_status for that loan via a prepared statement, ready to feed into the same EMI formula as 09_emi_calculation.sql.

All three scripts use mysqli prepared statements with bound parameters (bind_param), which mitigates SQL-injection risk; none currently escape output for HTML display, which is a lower-priority concern since these are backend data-entry endpoints rather than rendered pages.

7. Conclusion

Milestone 2 delivers a working three-layer slice of the loan-management domain, and it goes beyond CRUD scaffolding in a few places worth calling out: a genuine rule-based credit-eligibility engine with five ordered decision branches and a computed debt-to-income ratio, a resilient Angular service layer that degrades gracefully to local data when the backend is unreachable, and a validator library with real-world formats (PAN, Aadhaar, Indian mobile numbers). The backend team's own completion notes report a clean compile with no outstanding errors, and the DB layer ships both a MySQL/PHP reference implementation and reference SQL queries alongside the primary PostgreSQL/JPA path. The main open items before the next milestone are reconciling the two parallel database targets and duplicated enums/EMI logic described in Section 8, and broadening automated test coverage beyond the single existing spec file.

8. Team

Team Member	Role
Manikandan	Frontend Developer
Pavithra	Frontend Developer
Kousalya	Frontend Developer
Nithish	Backend Developer
Jeevana	Backend Developer
Raghvendra	Database Developer